

模块间服务接口契约梳理流程

Quantsociety Backend Project

2026 年 5 月 5 日

目标

规范各模块负责人梳理模块间服务接口的工作流程, 并产出模块目录下的 Markdown 接口文档: INTERFACE.md 或 SERVICE_CONTRACT.md。

接口文档必须明确:

1. 模块对外提供的封装服务入口;
2. 上游模块应调用的服务, 而非内部函数;
3. 每个服务的输入、输出、结果产物、Manifest、Event 和错误语义;
4. 禁止跨模块直接依赖的函数、类、脚本和内部包。

基本原则

1. 跨模块调用优先通过 Public Service Entry, 不直接 import 对方内部函数。
2. Public Function 不等于 Public Service Entry。
3. Backing Function 是服务内部实现细节, 不作为跨模块协作契约。
4. Job 型服务必须返回 run_id, 不得直接返回大体积结果。
5. 大体积结果通过 artifact_uri 或 manifest_uri 获取。
6. 未登记接口默认视为 Internal API。
7. 接口变更必须同步更新 Markdown 接口文档。

接口分层

层级	含义	跨模块调用建议
Internal Function	模块内部辅助函数、私有实现	禁止
Public Python API	稳定 Python 函数或类	谨慎允许
Public Service Entry	模块封装后的服务入口	推荐
Job API	异步任务入口	推荐
Query API	状态、摘要、结果查询入口	推荐
Event	模块间状态通知	推荐

Artifact / Manifest

结果产物与任务记录

必须规范

流程总览

步骤	任务	输出
1	明确模块职责边界	模块负责范围、不负责范围、上下游关系
2	盘点现有代码入口	函数、类、脚本、CLI、Job 入口清单
3	区分 Public 与 Internal	Public API 清单、Internal API 清单
4	设计 Public Service Entry	对外服务入口清单
5	定义 Request / Response	输入输出字段、类型、必填项
6	定义 Artifact / Manifest	结果路径、产物权限、运行记录
7	定义 Event	成功、失败、状态变化事件
8	定义错误语义	错误码、异常类型、是否可重试
9	标注调用边界	allowed callers、consumers、forbidden direct calls
10	产出 Markdown 文档	INTERFACE.md 或 SERVICE_CONTRACT.md

步骤一：明确模块职责边界

每个模块必须写清楚：

1. 本模块负责什么；
2. 本模块不负责什么；
3. 本模块消费哪些上游产物；
4. 本模块向哪些下游提供服务或结果；
5. 本模块是否产生可复用 artifact。

示例：

factor_layer 负责：

- 因子表达式定义
- 因子计算
- 因子落盘
- 因子评估
- 因子准入

factor_layer 不负责：

- 原始数据抓取
- 策略持仓生成
- 回测执行

- 前端展示

步骤二：盘点现有代码入口

模块负责人需要列出当前模块中可能被外部使用的入口：

1. Python 函数；
2. Python 类；
3. CLI 脚本；
4. demo 入口；
5. Job 入口；
6. 配置驱动入口；
7. 结果读取入口。

入口名称	类型	当前用途	是否建议对外
<code><module.function></code>	function	<code><usage></code>	yes / no / review
<code><module.Service></code>	class	<code><usage></code>	yes / no / review
<code><script.py></code>	script	<code><usage></code>	yes / no / review

步骤三：区分 Public 与 Internal

每个入口必须被标记为以下状态之一：

状态	含义	外部可调用
Stable	稳定接口，允许跨模块依赖	是
Experimental	试验接口，允许有限使用	谨慎
Internal	内部实现，不允许跨模块依赖	否
Deprecated	即将废弃，不建议继续使用	否

规则：

1. 未登记入口默认视为 Internal。
2. Internal API 不得被其他模块直接 import。
3. 如果下游确实需要 Internal 能力，应新增 Public Service Entry。

步骤四：设计 Public Service Entry

跨模块协作的主要对象是封装后的服务入口，而不是底层函数。

服务示例：

DataIngestService

FactorMaterializationService

FactorEvaluationService

StrategySignalService

BacktestService

ArtifactQueryService

Service	Method	Type	Allowed Callers
FactorMaterializationService	submit	job service	factor_agent, job_api
BacktestService	submit	job service	strategy_layer, job_api
ArtifactQueryService	get_manifest	query service	internal modules

步骤五：定义 Request 与 Response

每个 Public Service Entry 必须定义 request schema 与 response schema。

Request 至少包括：

1. 调用方；
2. 输入数据或配置路径；
3. 执行模式；
4. 输出可见性；
5. 参数字段；
6. 幂等性标识，若适用。

Response 至少包括：

1. run_id；
2. status；
3. manifest_uri；
4. artifact_uris；
5. error，若失败。

示例：

request:

```
requested_by: strategy_layer
input_uri: workspace_data/strategy/signals/xxx.parquet
run_mode: execute
output_visibility: internal
params:
  initial_cash: 100000
  commission: 0.001
```

response:

```
run_id: backtest_20260505_001
status: queued
manifest_uri: workspace_data/manifests/backtest_20260505_001.json
```

```
artifact_uris: []
```

步骤六：定义 Artifact 与 Manifest

服务不得只说明“返回结果”，必须说明结果落点、读取权限和追溯方式。

Artifact	Path / URI	Producer	Consumer
factor lake	workspace_data/factors/lake	factor service	strategy, back-test
backtest summary	workspace_data/backtests/...	backtest service	report, query api
manifest	workspace_data/manifests/...	job services	job api, audit

Manifest 至少包含：

```
run_id
service
method
requested_by
status
started_at
finished_at
input_uri
artifact_uris
allowed_consumers
error
git_commit
environment
```

步骤七：定义 Event

Job 型服务应声明成功、失败和关键状态变化事件。

事件命名建议：

```
<module>.<service>.<status>
```

示例：

```
factor.materialization.succeeded
factor.materialization.failed
backtest.run.succeeded
backtest.run.failed
data.ingest.completed
```

Event 字段至少包括：

```
event_name
run_id
producer
status
manifest_uri
artifact_uris
error
created_at
```

步骤八：定义错误语义

每个服务必须列出可能错误，并说明是否可重试。

错误码	类型	可重试	说明
CONFIG_INVALID	config	no	配置缺失或格式错误
DATA_NOT_FOUND	data	no	输入数据不存在
PERMISSION_DENIED	permission	no	调用方无权限
UPSTREAM_UNAVAILABLE	dependency	yes	上游服务不可用
EXECUTION_FAILED	runtime	maybe	执行失败，需查看日志

步骤九：标注调用边界

每个 Service Entry 必须声明：

1. allowed callers;
2. forbidden callers;
3. upstream dependencies;
4. downstream consumers;
5. forbidden direct imports。

示例：

```
allowed_callers:
```

- factor_agent
- job_api

```
downstream_consumers:
```

- strategy_layer
- backtest_layer

```
forbidden_direct_imports:
```

- factor_engine.expr.*
- factor_engine.ir.*
- factor_engine.planner.*

步骤十：产出 Markdown 接口文档

建议结构：

```
# <module_name> Interface Contract

## 1. Module Responsibility
## 2. Public Service Entries
## 3. Request / Response Contract
## 4. Backing Python APIs
## 5. Artifacts
## 6. Manifest Contract
## 7. Events
## 8. Error Contract
## 9. Upstream / Downstream
## 10. Forbidden Direct Calls
## 11. Compatibility Policy
```

提交要求

每个模块负责人提交时至少包含：

1. 一个 INTERFACE.md 或 SERVICE_CONTRACT.md；
2. Public Service Entry 表；
3. 每个服务的 request / response；
4. artifact 与 manifest 说明；
5. allowed callers 与 forbidden direct calls；
6. unresolved issues。

Review Checklist

完成	检查项
☐	是否明确模块职责与不负责范围
☐	是否列出 Public Service Entry
☐	是否区分 Public API 与 Internal API
☐	是否定义 request schema
☐	是否定义 response schema

- ☐ 是否定义 artifact path / URI
 - ☐ 是否定义 manifest contract
 - ☐ 是否定义 event contract
 - ☐ 是否定义 error contract
 - ☐ 是否列出 allowed callers
 - ☐ 是否列出 forbidden direct calls
 - ☐ 是否说明 backing functions
 - ☐ 是否避免直接暴露内部实现
 - ☐ 是否避免 response 直接返回大体积结果
 - ☐ 是否标注 unresolved issues
-